CRYSTAL

Documentation des Requetes d'Audit

REQUETES MONGODB — 10 CATEGORIES — 37 CONTROLES

ARIEL INDUSTRIES SARL

Version 1.0 - Mai 2026

Confidentiel

SOMMAIRE

Cat.	Titre	Type	Controles
1	INTÉGRITÉ DES DONNÉES	INCONDITIONNEL	4
2	CONTRÔLES SUR LES BÉNÉFICIAIRES	CONDITIONNEL	6
3	CONTRÔLES SUR LES PRESTATAIRES	CONDITIONNEL	5
4	CONTRÔLES DE DOUBLONS	INCONDITIONNEL	4
5	FRÉQUENCE ET VOLUMÉTRIE	INCONDITIONNEL	5
6	CONTRÔLES SUR LES MONTANTS	CONDITIONNEL	6
7	CONTRÔLES SUR LES ACTES MÉDICAUX	CONDITIONNEL	5
8	CONTRÔLES TEMPORELS	INCONDITIONNEL	5
9	COHÉRENCE CONTRACTUELLE	CONDITIONNEL	4
10	ANALYSES STATISTIQUES	INCONDITIONNEL	5

CATÉGORIE 1 — INTÉGRITÉ DES DONNÉES

INCONDITIONNEL

Vérifie la qualité et la cohérence de base des données avant tout autre contrôle. Détecte les champs manquants, les formats invalides et les montants non numériques.

1.1 — Champs obligatoires manquants

Objectif : *Détecte toute ligne dont au moins un champ critique est vide ou null : dateSoins, dateComptable, matriculeAssurePrincipal, matriculeBeneficiaire, montants.*

```
// Champs critiques à vérifier const champsObligatoires = [
'dateSoins','dateComptable','matriculeAssurePrincipal',
'matriculeBeneficiaire','montantReclame','montantExclus', 'baseRembourse','montantTM','montantPaye' ];
// Filtre $or dynamique couvrant null, undefined et chaîne vide const conditions =
champsObligatoires.map(champ => ({ [champ]: { $in: [null, undefined, ''] } })); const lignes = await
Prestation.find({ fichierImporteId, $or: conditions }).select('numeroPrestation matriculeAssurePrincipal
nomAssurePrincipal prenomAssurePrincipal matriculeBeneficiaire nomBeneficiaire prenomBeneficiaire
dateSoins dateComptable montantPaye');
```

1.2 — Formats de dates invalides

Objectif : *Détecte les dateSoins et dateComptable non reconnues comme dates valides (chaînes non parsables ou dates avec timestamp NaN).*

```
// Détection des dates stockées comme chaînes (non converties) const lignesType = await Prestation.find({
fichierImporteId, $or: [ { dateSoins: { $type: 'string' } }, { dateComptable: { $type: 'string' } } ] });
// Vérification supplémentaire JS pour les dates invalides (NaN) const lignesNaN = (await
Prestation.find({ fichierImporteId })) .filter(l => { const dsInvalide = l.dateSoins && isNaN(new
Date(l.dateSoins).getTime()); const dcInvalide = l.dateComptable && isNaN(new
Date(l.dateComptable).getTime()); return dsInvalide || dcInvalide; });
```

1.3 — Matricules absents

Objectif : *Vérifie que matriculeAssurePrincipal et matriculeBeneficiaire sont renseignés. Aucune vérification de conformité au référentiel à ce stade.*

```
const lignes = await Prestation.find({ fichierImporteId, $or: [ { matriculeAssurePrincipal: { $in: [null,
undefined, ''] } }, { matriculeBeneficiaire: { $in: [null, undefined, ''] } } ]
}).select('numeroPrestation matriculeAssurePrincipal nomAssurePrincipal prenomAssurePrincipal
matriculeBeneficiaire nomBeneficiaire prenomBeneficiaire montantPaye');
```

1.4 — Valeurs montants non numériques

Objectif : *Vérifie que tous les champs financiers contiennent des valeurs numériques >= 0. Détecte les chaînes texte et les valeurs négatives.*

```
const champsMonetaires = [ 'montantReclame','montantExclus', 'baseRembourse','montantTM','montantPaye'
]; // $type:'string' = non converti | $lt:0 = valeur négative const conditions =
champsMonetaires.flatMap(champ => [ { [champ]: { $type: 'string' } }, { [champ]: { $lt: 0 } } ]); const
lignes = await Prestation.find({ fichierImporteId, $or: conditions }).select('numeroPrestation
matriculeAssurePrincipal nomAssurePrincipal prenomAssurePrincipal matriculeBeneficiaire nomBeneficiaire
prenomBeneficiaire montantReclame montantExclus baseRembourse montantTM montantPaye');
```

CATÉGORIE 2 — CONTRÔLES SUR LES BÉNÉFICIAIRES

CONDITIONNEL | *Condition : Table refAssures renseignée*

Croise les données du fichier de soins avec le référentiel des assurés. Détecte les bénéficiaires inexistants, radiés, non affiliés ou hors contrat.

2.1 — Bénéficiaire inexistant dans le référentiel

Objectif : *$lookup vers refAssures — toute prestation sans correspondance matricule est une anomalie critique.*

```
await Prestation.aggregate([ { $match: { fichierImporteId } }, { $lookup: { from: 'refassures',
localField: 'matriculeBeneficiaire', foreignField: 'matricule', pipeline: [{ $match: { compagnieId } }],
as: 'assureRef' }}, // Aucune correspondance = bénéficiaire inexistant { $match: { assureRef: { $size: 0
} } }, { $project: { numeroPrestation:1, matriculeAssurePrincipal:1, nomAssurePrincipal:1,
prenomAssurePrincipal:1, matriculeBeneficiaire:1, nomBeneficiaire:1, prenomBeneficiaire:1, dateSoins:1,
montantPaye:1 }} ]);
```

2.2 — Bénéficiaire radié

Objectif : *dateSoins > dateRadiation dans refAssures → soins postérieurs à la radiation (fraude).*

```
await Prestation.aggregate([ { $match: { fichierImporteId } }, { $lookup: { from: 'refassures',
localField: 'matriculeBeneficiaire', foreignField: 'matricule', pipeline: [{ $match: { compagnieId,
dateRadiation: { $ne: null } } }], as: 'assureRef' }}, { $unwind: '$assureRef' }, // dateSoins
strictement après la date de radiation { $match: { $expr: { $gt: ['$dateSoins',
'$assureRef.dateRadiation'] } }}, { $project: { matriculeBeneficiaire:1, nomBeneficiaire:1,
prenomBeneficiaire:1, nomAssurePrincipal:1, prenomAssurePrincipal:1, matriculeAssurePrincipal:1,
dateSoins:1, montantPaye:1, dateRadiation: '$assureRef.dateRadiation' }} ]);
```

2.3 — Bénéficiaire non encore affilié

Objectif : *dateSoins < dateAffiliation dans refAssures → soins antérieurs à l'affiliation.*

```
await Prestation.aggregate([ { $match: { fichierImporteId } }, { $lookup: { from: 'refassures',
localField: 'matriculeBeneficiaire', foreignField: 'matricule', pipeline: [{ $match: { compagnieId } }],
as: 'assureRef' }}, { $unwind: '$assureRef' }, { $match: { $expr: { $lt: ['$dateSoins',
'$assureRef.dateAffiliation'] } }}, { $project: { matriculeBeneficiaire:1, nomBeneficiaire:1,
prenomBeneficiaire:1, nomAssurePrincipal:1, prenomAssurePrincipal:1, matriculeAssurePrincipal:1,
dateSoins:1, montantPaye:1, dateAffiliation: '$assureRef.dateAffiliation' }} ]);
```

2.4 — Incohérence nom/matricule

Objectif : *Comparaison insensible à la casse du nom et prénom du bénéficiaire entre le fichier et refAssures.*

```
await Prestation.aggregate([ { $match: { fichierImporteId } }, { $lookup: { from: 'refassures',
localField: 'matriculeBeneficiaire', foreignField: 'matricule', pipeline: [{ $match: { compagnieId } }],
as: 'assureRef' }}, { $unwind: { path:'$assureRef', preserveNullAndEmpty:false }}, // Ni le nom ni le
prénom ne correspondent (toLower + trim) { $match: { $expr: { $and: [ { $ne: [{
$toLower:{$trim:{input:'$nomBeneficiaire'}} }, { $toLower:{$trim:{input:'$assureRef.nom'}} }] }, { $ne:
[{ $toLower:{$trim:{input:'$prenomBeneficiaire'}} }, { $toLower:{$trim:{input:'$assureRef.prenom'}} }] }
]}}}, { $project: { matriculeBeneficiaire:1, nomBeneficiaire:1, prenomBeneficiaire:1,
nomAssurePrincipal:1, prenomAssurePrincipal:1, matriculeAssurePrincipal:1, nomRef:'$assureRef.nom',
prenomRef:'$assureRef.prenom', dateSoins:1, montantPaye:1 }} ]);
```

2.5 — Bénéficiaire hors contrat

Objectif : *Vérifie que le collège et le numéro de contrat de la prestation correspondent à ceux de l'assuré dans refAssures.*

```
await Prestation.aggregate([ { $match: { fichierImporteId } }, { $lookup: { from: 'refassures',
localField: 'matriculeBeneficiaire', foreignField: 'matricule', pipeline: [{ $match: { compagnieId } }],
as: 'assureRef' }}, { $unwind: { path:'$assureRef', preserveNullAndEmpty:false }}, { $match: { $expr: {
$or: [ // Collège incohérent { $and: [ { $ne:['$college','$assureRef.college'] }, {
$ne:['$assureRef.college', null] } ]}, // Contrat incohérent { $and: [ {
$ne:['$numeroContrat','$assureRef.numeroContrat'] }, { $ne:['$assureRef.numeroContrat', null] } ]} ]}}},
{ $project: { matriculeBeneficiaire:1, nomBeneficiaire:1, prenomBeneficiaire:1, nomAssurePrincipal:1,
prenomAssurePrincipal:1, matriculeAssurePrincipal:1, college:1, numeroContrat:1,
collegeRef:'$assureRef.college', contratRef:'$assureRef.numeroContrat', dateSoins:1, montantPaye:1 }}
]);
```

2.6 — Ayant droit dépassant l'âge limite

Objectif : *Calcule l'âge du bénéficiaire (ayant droit) à la dateSoins via $dateDiff. Alerte si âge > limite contractuelle (défaut : 21 ans).*

```
// ageLimite = paramètre configurable (défaut 21 ans) await Prestation.aggregate([ { $match: {
fichierImporteId } }, { $lookup: { from: 'refassures', localField: 'matriculeBeneficiaire',
foreignField: 'matricule', pipeline: [{ $match: { compagnieId, typeAssure: 'ayant_droit', dateNaissance:
{ $ne: null } }}], as: 'assureRef' }}, { $unwind: { path:'$assureRef', preserveNullAndEmpty:false }}, //
Calcul de l'âge en années avec $dateDiff (MongoDB 5.0+) { $addFields: { ageLorsSoins: { $dateDiff: {
startDate: '$assureRef.dateNaissance', endDate: '$dateSoins', unit: 'year' } } }}, // Conserver les
bénéficiaires dépassant l'âge limite { $match: { ageLorsSoins: { $gt: ageLimite } } }, { $project: {
matriculeBeneficiaire:1, nomBeneficiaire:1, prenomBeneficiaire:1, nomAssurePrincipal:1,
prenomAssurePrincipal:1, matriculeAssurePrincipal:1, dateSoins:1, montantPaye:1, ageLorsSoins:1,
dateNaissance:'$assureRef.dateNaissance' }} ]);
```

CATÉGORIE 3 — CONTRÔLES SUR LES PRESTATAIRES

CONDITIONNEL | *Condition : Table refPrestataires renseignée*

Vérifie la légitimité des prestataires ayant dispensé les soins : agrément, statut, cohérence des données et habilitation des prescripteurs.

3.1 — Prestataire non agréé

Objectif : *codePrestataire absent de la collection refPrestataires de la compagnie → hors réseau agréé.*

```
await Prestation.aggregate([ { $match: { fichierImporteId } }, { $lookup: { from: 'refprestataires', localField: 'codePrestataire', foreignField: 'code', pipeline: [{ $match: { compagnieId } }], as: 'prestRef' }}, // Taille 0 = absent du réseau agréé { $match: { prestRef: { $size: 0 } } }, { $project: { numeroPrestation:1, matriculeAssurePrincipal:1, nomAssurePrincipal:1, prenomAssurePrincipal:1, matriculeBeneficiaire:1, nomBeneficiaire:1, prenomBeneficiaire:1, codePrestataire:1, prestataire:1, dateSoins:1, montantPaye:1 }} ]);
```

3.2 — Prestataire radié ou suspendu

Objectif : *Prestataire avec statut 'radié'/'suspendu' dans refPrestataires, ou dont la dateSoins est postérieure à sa dateRadiation.*

```
await Prestation.aggregate([ { $match: { fichierImporteId } }, { $lookup: { from: 'refprestataires', localField: 'codePrestataire', foreignField: 'code', pipeline: [{ $match: { compagnieId, $or: [ { statut: { $in: ['radie','suspendu'] } }, { dateRadiation: { $ne: null } } ] }}], as: 'prestRef' }}, { $unwind: { path:'$prestRef', preserveNullAndEmpty:false }}, // Statut radié/suspendu OU soins après dateRadiation { $match: { $or: [ { 'prestRef.statut': { $in: ['radie','suspendu'] } }, { $expr: { $and: [ { $ne:['$prestRef.dateRadiation', null] }, { $gt:['$dateSoins','$prestRef.dateRadiation'] } ]}} ]}}, { $project: { matriculeBeneficiaire:1, nomBeneficiaire:1, prenomBeneficiaire:1, nomAssurePrincipal:1, prenomAssurePrincipal:1, matriculeAssurePrincipal:1, codePrestataire:1, prestataire:1, dateSoins:1, montantPaye:1, statutPrest:'$prestRef.statut', dateRadiation:'$prestRef.dateRadiation' }} ]);
```

3.3 — Incohérence code/nom prestataire

Objectif : *Le nom du prestataire dans le fichier diffère du nom dans refPrestataires pour le même code.*

```
await Prestation.aggregate([ { $match: { fichierImporteId } }, { $lookup: { from: 'refprestataires', localField: 'codePrestataire', foreignField: 'code', pipeline: [{ $match: { compagnieId } }], as: 'prestRef' }}, { $unwind: { path:'$prestRef', preserveNullAndEmpty:false }}, // Comparaison insensible à la casse via $toLower { $match: { $expr: { $ne: [ { $toLower:{ $trim:{ input:'$prestataire' } } }, { $toLower:{ $trim:{ input:'$prestRef.nom' } } } ]}}}, { $project: { matriculeBeneficiaire:1, nomBeneficiaire:1, prenomBeneficiaire:1, nomAssurePrincipal:1, prenomAssurePrincipal:1, matriculeAssurePrincipal:1, codePrestataire:1, prestataire:1, nomRef:'$prestRef.nom', dateSoins:1, montantPaye:1 }} ]);
```

3.4 — Prestataire géographiquement incohérent

Objectif : *Zone du prestataire différente de la zone de l'assuré, hors actes d'hospitalisation et d'urgence.*

```
await Prestation.aggregate([ { $match: { fichierImporteId, famillePrestation: { $nin: ['HOSPITALISATION','URGENCE'] } }}, { $lookup: { from: 'refprestataires', localField: 'codePrestataire', foreignField: 'code', pipeline: [{ $match: { compagnieId } }], as: 'prestRef' }}, { $unwind: { path:'$prestRef', preserveNullAndEmpty:false }}, { $lookup: { from: 'refassures', localField: 'matriculeBeneficiaire', foreignField: 'matricule', pipeline: [{ $match: { compagnieId, zone:{ $ne:null } } }], as: 'assureRef' }}, { $unwind: { path:'$assureRef', preserveNullAndEmpty:false }}, // Zone prestataire != zone assuré { $match: { $expr: { $and: [ { $ne:['$prestRef.zone', null] }, { $ne:['$assureRef.zone', null] }, { $ne:['$prestRef.zone','$assureRef.zone'] } ]}}}, { $project: { matriculeBeneficiaire:1, nomBeneficiaire:1, prenomBeneficiaire:1, nomAssurePrincipal:1, prenomAssurePrincipal:1, matriculeAssurePrincipal:1, codePrestataire:1, prestataire:1, zonePrest:'$prestRef.zone', zoneAssure:'$assureRef.zone', dateSoins:1, montantPaye:1 }} ]);
```

3.5 — Prescripteur non habilité

Objectif : *Le codePrescripteur absent de refPrescripteurs, ou la famille de prestation n'est pas dans ses familles autorisées.*

```
await Prestation.aggregate([ { $match: { fichierImporteId, codePrescripteur: { $nin:[null,''] } }}, {
$lookup: { from: 'refprescripteurs', localField: 'codePrescripteur', foreignField: 'code', pipeline: [{
$match: { compagnieId } }], as: 'prescRef' }}, { $match: { $or: [ // Prescripteur totalement absent {
prescRef: { $size: 0 } }, // Présent mais famille non autorisée { $expr: { $and: [ { $gt:[{
$size:'$prescRef' }, 0] }, { $not: { $in: [ '$famillePrestation', { $ifNull:[ {
$arrayElemAt:['$prescRef.famillesAutorisees',0] }, [] ]} ]}} ]}} ]}}, { $project: {
matriculeBeneficiaire:1, nomBeneficiaire:1, prenomBeneficiaire:1, nomAssurePrincipal:1,
prenomAssurePrincipal:1, matriculeAssurePrincipal:1, codePrescripteur:1, nomPrescripteur:1,
famillePrestation:1, dateSoins:1, montantPaye:1 }} ]);
```

CATÉGORIE 4 — CONTRÔLES DE DOUBLONS

INCONDITIONNEL

Détecte les prestations facturées plusieurs fois : doublons stricts sur le numéro de prestation, doublons fonctionnels sur la clé métier, et doublons entre gestionnaires ou entre mois.

4.1 — Doublon strict

Objectif : *Même numeroPrestation apparaissant plusieurs fois dans le même fichier.*

```
await Prestation.aggregate([ { $match: { fichierImporteId, numeroPrestation: { $nin:[null,''] } }}, { $group: { _id: '$numeroPrestation', count: { $sum: 1 }, ids: { $push: '$_id' }, montantTotal: { $sum: '$montantPaye' }, nomBeneficiaire: { $first: '$nomBeneficiaire' }, prenomBeneficiaire: { $first: '$prenomBeneficiaire' }, nomAssurePrincipal: { $first: '$nomAssurePrincipal' }, prenomAssurePrincipal: { $first: '$prenomAssurePrincipal' }, matriculeAssurePrincipal: { $first: '$matriculeAssurePrincipal' }, matriculeBeneficiaire: { $first: '$matriculeBeneficiaire' }, dateSoins: { $first: '$dateSoins' } }}, // Garder uniquement les groupes avec plus d'une occurrence { $match: { count: { $gt: 1 } } } ]);
```

4.2 — Doublon fonctionnel

Objectif : *Même combinaison clé métier : matriculeBeneficiaire + dateSoins + codeActe + codePrestataire + montantPaye → prestation facturée deux fois.*

```
await Prestation.aggregate([ { $match: { fichierImporteId } }, { $group: { _id: { matriculeBeneficiaire: '$matriculeBeneficiaire', dateSoins: '$dateSoins', codeActe: '$codeActe', codePrestataire: '$codePrestataire', montantPaye: '$montantPaye' }, count: { $sum: 1 }, ids: { $push: '$_id' }, montantTotal: { $sum: '$montantPaye' }, nomBeneficiaire: { $first: '$nomBeneficiaire' }, prenomBeneficiaire: { $first: '$prenomBeneficiaire' }, nomAssurePrincipal: { $first: '$nomAssurePrincipal' }, prenomAssurePrincipal: { $first: '$prenomAssurePrincipal' }, matriculeAssurePrincipal: { $first: '$matriculeAssurePrincipal' }, prestataire: { $first: '$prestataire' } }}, // Montant en trop = total - 1 occurrence légitime { $match: { count: { $gt: 1 } } } ]);
```

4.3 — Doublon inter-gestionnaires

Objectif : *Même clé métier facturée par deux gestionnaires différents pour la même compagnie.*

```
// Pour chaque prestation du fichier courant, // chercher une correspondance dans d'autres fichiers // d'un gestionnaire différent de la même compagnie const doublon = await Prestation.findOne({ fichierImporteId: { $ne: fichierImporteId }, // Autre fichier compagnieId, // Même compagnie gestionnaireId: { $ne: gestionnaireId }, // Autre gestionnaire matriculeBeneficiaire: prestation.matriculeBeneficiaire, dateSoins: prestation.dateSoins, codeActe: prestation.codeActe, codePrestataire: prestation.codePrestataire, montantPaye: prestation.montantPaye }).select('gestionnaireId fichierImporteId');
```

4.4 — Doublon inter-mois

Objectif : *Même clé métier déjà facturée dans un fichier précédent du même gestionnaire.*

```
// Recherche dans les fichiers antérieurs du même gestionnaire const doublon = await Prestation.findOne({ fichierImporteId: { $ne: fichierImporteId }, // Autre fichier gestionnaireId, // Même gestionnaire matriculeBeneficiaire: prestation.matriculeBeneficiaire, dateSoins: prestation.dateSoins, codeActe: prestation.codeActe, codePrestataire: prestation.codePrestataire, montantPaye: prestation.montantPaye }).select('fichierImporteId');
```

CATÉGORIE 5 — FRÉQUENCE ET VOLUMÉTRIE

INCONDITIONNEL

Détecte les comportements statistiquement anormaux : surconcentration de consultations, présence dans plusieurs centres le même jour, quantités pharmaceutiques excessives et pics de fin de période.

5.1 — Fréquence de consultations anormale

Objectif : *Groupe les consultations par bénéficiaire et par mois. Alerte si le nombre dépasse le seuil configurable (défaut : 10/mois).*

```
// seuil = options.maxConsultationsMois || 10 await Prestation.aggregate([ { $match: { fichierImporteId, famillePrestation: { $in: [ 'CONSULTATION','MEDECINE_GENERALE','SPECIALISTE' ]} }}, { $group: { _id: { matriculeBeneficiaire: '$matriculeBeneficiaire', mois: { $month: '$dateSoins' }, annee: { $year: '$dateSoins' } }, count: { $sum: 1 }, montantTotal: { $sum: '$montantPaye' }, nomBeneficiaire: { $first: '$nomBeneficiaire' }, prenomBeneficiaire: { $first: '$prenomBeneficiaire' }, nomAssurePrincipal: { $first: '$nomAssurePrincipal' }, prenomAssurePrincipal: { $first: '$prenomAssurePrincipal' }, matriculeAssurePrincipal: { $first: '$matriculeAssurePrincipal' }, ids: { $push: '$_id' } }}, { $match: { count: { $gt: seuil } } } ]);
```

5.2 — Fréquence chez le même prestataire

Objectif : *Groupe par bénéficiaire + prestataire + mois. Alerte si visites > seuil configurable (défaut : 8/mois).*

```
// seuil = options.maxVisitesMemePrestataire || 8 await Prestation.aggregate([ { $match: { fichierImporteId } }, { $group: { _id: { matriculeBeneficiaire: '$matriculeBeneficiaire', codePrestataire: '$codePrestataire', mois: { $month: '$dateSoins' }, annee: { $year: '$dateSoins' } }, count: { $sum: 1 }, montantTotal: { $sum: '$montantPaye' }, nomBeneficiaire: { $first: '$nomBeneficiaire' }, prenomBeneficiaire: { $first: '$prenomBeneficiaire' }, nomAssurePrincipal: { $first: '$nomAssurePrincipal' }, prenomAssurePrincipal: { $first: '$prenomAssurePrincipal' }, matriculeAssurePrincipal: { $first: '$matriculeAssurePrincipal' }, prestataire: { $first: '$prestataire' }, ids: { $push: '$_id' } }}, { $match: { count: { $gt: seuil } } } ]);
```

5.3 / 5.4 — Même jour, plusieurs prestataires

Objectif : *Groupe par bénéficiaire + date (YYYY-MM-DD). Détecte les jours où le bénéficiaire est présent dans plusieurs centres. Signale aussi les actes incompatibles (ex: hospitalisation + consultation externe).*

```
await Prestation.aggregate([ { $match: { fichierImporteId } }, { $group: { _id: { matriculeBeneficiaire: '$matriculeBeneficiaire', dateSoins: { $dateToString:{ format:'%Y-%m-%d', date:'$dateSoins' } } }, prestatairesDistincts: { $addToSet: '$codePrestataire' }, famillesDistinctes: { $addToSet: '$famillePrestation' }, montantTotal: { $sum: '$montantPaye' }, nomBeneficiaire: { $first: '$nomBeneficiaire' }, prenomBeneficiaire: { $first: '$prenomBeneficiaire' }, nomAssurePrincipal: { $first: '$nomAssurePrincipal' }, prenomAssurePrincipal: { $first: '$prenomAssurePrincipal' }, matriculeAssurePrincipal: { $first: '$matriculeAssurePrincipal' }, ids: { $push: '$_id' } }}, // Plus d'un prestataire distinct le même jour { $match: { $expr: { $gt:[{ $size:'$prestatairesDistincts' }, 1] } }} ]); // Actes incompatibles = HOSPITALISATION + autre famille le même jour
```

5.5 — Volumétrie pharmaceutique anormale

Objectif : *Détecte les ordonnances pharmaceutiques avec quantité > seuil configurable (défaut : 10 unités).*

```
// seuil = options.maxQuantiteMedicament || 10 await Prestation.find({ fichierImporteId, famillePrestation: 'PHARMACIE', quantite: { $gt: seuil } }).select('numeroPrestation matriculeAssurePrincipal nomAssurePrincipal prenomAssurePrincipal matriculeBeneficiaire nomBeneficiaire prenomBeneficiaire codeActe quantite montantPaye dateSoins prestataire');
```

5.6 — Pic de consommation en fin de période

Objectif : *Alerte si > 40% des montants sont concentrés dans les 5 derniers jours du mois (seuil configurable). Utilise $dayOfMonth et $dateDiff.*

```
// Calcul du montant des 5 derniers jours du mois await Prestation.aggregate([ { $match: {
fichierImporteId } }, { $addFields: { jourDuMois: { $dayOfMonth: '$dateSoins' }, dernierJourMois: {
$dayOfMonth: { $dateFromParts: { year: { $year: '$dateSoins' }, month: { $add:[{ $month:'$dateSoins' },
1] }, day: 0 }}} }}, // Garder les 5 derniers jours du mois { $match: { $expr: { $gte: [ '$jourDuMois', {
$subtract:['$dernierJourMois', 4] } ]}}}, { $group: { _id: null, montantFinPeriode: { $sum:
'$montantPaye' }, nbLignes: { $sum: 1 } }} ]); // Alerte si montantFinPeriode / montantTotal > seuilRatio
(0.40)
```

CATÉGORIE 6 — CONTRÔLES SUR LES MONTANTS

CONDITIONNEL | *Condition : Tables refActes et/ou refPlafonds renseignées*

Vérifie les tarifs, l'application du ticket modérateur et les dépassements de plafonds (par acte, par assuré mensuel/annuel, par collège).

6.1 — Montant réclamé hors tarif conventionné

Objectif : *montantReclame > tarifConventionne dans refActes pour le même codeActe.*

```
await Prestation.aggregate([ { $match: { fichierImporteId } }, { $lookup: { from: 'refactes', localField:
'codeActe', foreignField: 'codeActe', pipeline: [{ $match: { compagnieId, tarifConventionne:{ $gt:0 } }
}], as: 'acteRef' }}, { $unwind:{ path:'$acteRef', preserveNullAndEmpty:false }}, { $match: { $expr: {
$gt:['$montantReclame','$acteRef.tarifConventionne'] } }}, { $project: { matriculeBeneficiaire:1,
nomBeneficiaire:1, prenomBeneficiaire:1, nomAssurePrincipal:1, prenomAssurePrincipal:1,
matriculeAssurePrincipal:1, codeActe:1, montantReclame:1, montantPaye:1,
tarifConventionne:'$acteRef.tarifConventionne', depassement:{ $subtract:['$montantReclame',
'$acteRef.tarifConventionne'] } }} ]);
```

6.2 — Ticket modérateur non appliqué

Objectif : *montantTM = 0 alors que refActes prévoit un taux de ticket modérateur > 0 pour cet acte.*

```
await Prestation.aggregate([ { $match: { fichierImporteId, $or:[{ montantTM:0 },{ montantTM:null }] }}, {
$lookup: { from: 'refactes', localField: 'codeActe', foreignField: 'codeActe', pipeline: [{ $match: {
compagnieId, tauxTicketModerateur:{ $gt:0 } } }], as: 'acteRef' }}, { $unwind:{ path:'$acteRef',
preserveNullAndEmpty:false }}, { $project: { matriculeBeneficiaire:1, nomBeneficiaire:1,
prenomBeneficiaire:1, nomAssurePrincipal:1, prenomAssurePrincipal:1, matriculeAssurePrincipal:1,
codeActe:1, montantPaye:1, montantTM:1, tauxTM:'$acteRef.tauxTicketModerateur', // TM attendu =
montantPaye * taux / 100 tmAttendu:{ $multiply:['$montantPaye', {
$divide:['$acteRef.tauxTicketModerateur',100] }] } }} ]);
```

6.3 — Dépassement du plafond par acte

Objectif : *montantPaye > plafondUnitaire dans refActes pour le même codeActe.*

```
await Prestation.aggregate([ { $match: { fichierImporteId } }, { $lookup: { from: 'refactes', localField:
'codeActe', foreignField: 'codeActe', pipeline: [{ $match: { compagnieId, plafondUnitaire:{ $gt:0 } } }],
as: 'acteRef' }}, { $unwind:{ path:'$acteRef', preserveNullAndEmpty:false }}, { $match: { $expr: {
$gt:['$montantPaye','$acteRef.plafondUnitaire'] } }}, { $project: { matriculeBeneficiaire:1,
nomBeneficiaire:1, prenomBeneficiaire:1, nomAssurePrincipal:1, prenomAssurePrincipal:1,
matriculeAssurePrincipal:1, codeActe:1, montantPaye:1, plafondUnitaire:'$acteRef.plafondUnitaire',
depassement:{ $subtract:['$montantPaye', '$acteRef.plafondUnitaire'] } }} ]);
```

6.4 — Dépassement du plafond individuel mensuel

Objectif : *Cumul mensuel des montantPaye par bénéficiaire > plafondIndividuelMensuel dans refPlafonds.*

```
await Prestation.aggregate([ { $match: { fichierImporteId } }, // Cumul mensuel par bénéficiaire {
$group: { _id: { matriculeBeneficiaire: '$matriculeBeneficiaire', numeroContrat: '$numeroContrat', mois:
{ $month: '$dateSoins' }, annee: { $year: '$dateSoins' } }, cumulMensuel: { $sum: '$montantPaye' },
nomBeneficiaire: { $first: '$nomBeneficiaire' }, prenomBeneficiaire: { $first: '$prenomBeneficiaire' },
nomAssurePrincipal: { $first: '$nomAssurePrincipal' }, prenomAssurePrincipal: { $first:
'$prenomAssurePrincipal' }, matriculeAssurePrincipal: { $first: '$matriculeAssurePrincipal' }, ids: {
$push: '$_id' } }}, // Joindre le plafond mensuel de l'assuré { $lookup: { from: 'refplafonds',
localField: '_id.matriculeBeneficiaire', foreignField: 'matriculeBeneficiaire', pipeline: [{ $match: {
compagnieId, plafondIndividuelMensuel:{ $gt:0 } } }], as: 'plafondRef' }}, { $unwind:{
path:'$plafondRef', preserveNullAndEmpty:false }}, // Cumul > plafond mensuel { $match: { $expr:{
$gt:['$cumulMensuel', '$plafondRef.plafondIndividuelMensuel'] } }}, { $project: { cumulMensuel:1,
plafondMensuel:'$plafondRef.plafondIndividuelMensuel', depassement:{ $subtract:['$cumulMensuel',
'$plafondRef.plafondIndividuelMensuel'] }, nomBeneficiaire:1, prenomBeneficiaire:1,
nomAssurePrincipal:1, prenomAssurePrincipal:1, matriculeAssurePrincipal:1, ids:1 }} ]);
```

6.5 — Dépassement du plafond annuel individuel

Objectif : *Cumul annuel multi-fichiers par bénéficiaire > plafondIndividuelAnnuel. Agrège tous les fichiers de la compagnie pour l'année.*

```
// Cumul annuel sur TOUS les fichiers de la compagnie await Prestation.aggregate([ { $match: {
compagnieId, dateSoins: { $gte: new Date(`${annee}-01-01`), $lte: new Date(`${annee}-12-31T23:59:59`) }
}}, { $group: { _id: '$matriculeBeneficiaire', cumulAnnuel: { $sum: '$montantPaye' }, nomBeneficiaire: {
$first: '$nomBeneficiaire' }, prenomBeneficiaire: { $first: '$prenomBeneficiaire' }, nomAssurePrincipal:
{ $first: '$nomAssurePrincipal' }, prenomAssurePrincipal: { $first: '$prenomAssurePrincipal' },
matriculeAssurePrincipal: { $first: '$matriculeAssurePrincipal' } }}, { $lookup: { from: 'refplafonds',
localField: '_id', foreignField: 'matriculeBeneficiaire', pipeline: [{ $match: { compagnieId,
plafondIndividuelAnnuel:{ $gt:0 } } }], as: 'plafondRef' }}, { $unwind:{ path:'$plafondRef',
preserveNullAndEmpty:false }}, { $match: { $expr:{ $gt:['$cumulAnnuel',
'$plafondRef.plafondIndividuelAnnuel'] } }}, { $project: { cumulAnnuel:1,
plafondAnnuel:'$plafondRef.plafondIndividuelAnnuel', depassement:{ $subtract:['$cumulAnnuel',
'$plafondRef.plafondIndividuelAnnuel'] }, nomBeneficiaire:1, prenomBeneficiaire:1, nomAssurePrincipal:1,
prenomAssurePrincipal:1, matriculeAssurePrincipal:1 }} ]);
```

6.6 — Dépassement du plafond du collège

Objectif : *Cumul des montantPaye par collège dans le fichier > plafondCollege dans refPlafonds.*

```
await Prestation.aggregate([ { $match: { fichierImporteId } }, // Cumul par collège { $group: { _id:
'$college', numeroContrat: { $first: '$numeroContrat' }, cumulCollege: { $sum: '$montantPaye' },
nbBeneficiaires:{ $addToSet: '$matriculeBeneficiaire' }, ids: { $push: '$_id' } }}, { $lookup: { from:
'refplafonds', localField: '_id', foreignField: 'college', pipeline: [{ $match: { compagnieId,
plafondCollege:{ $gt:0 } } }], as: 'plafondRef' }}, { $unwind:{ path:'$plafondRef',
preserveNullAndEmpty:false }}, { $match: { $expr:{ $gt:['$cumulCollege','$plafondRef.plafondCollege'] }
}}, { $project: { college:'$_id', cumulCollege:1, ids:1, nbBeneficiaires:{ $size:'$nbBeneficiaires' },
numeroContrat:1, plafondCollege:'$plafondRef.plafondCollege', depassement:{ $subtract:['$cumulCollege',
'$plafondRef.plafondCollege'] } }} ]);
```

CATÉGORIE 7 — CONTRÔLES SUR LES ACTES MÉDICAUX

CONDITIONNEL | *Condition : Table refActes renseignée*

Vérifie la validité et la cohérence des actes médicaux : nomenclature, couverture contractuelle, compatibilité avec le profil du bénéficiaire et exigences d'hospitalisation.

7.1 — Acte inexistant dans la nomenclature

Objectif : *codeActe absent de la collection refActes → acte hors nomenclature.*

```
await Prestation.aggregate([ { $match: { fichierImporteId } }, { $lookup: { from: 'refactes', localField:
'codeActe', foreignField: 'codeActe', pipeline: [{ $match: { compagnieId } }], as: 'acteRef' }}, {
$match: { acteRef: { $size: 0 } } }, { $project: { matriculeBeneficiaire:1, nomBeneficiaire:1,
prenomBeneficiaire:1, nomAssurePrincipal:1, prenomAssurePrincipal:1, matriculeAssurePrincipal:1,
codeActe:1, famillePrestation:1, dateSoins:1, montantPaye:1 }} ]);
```

7.2 — Acte non couvert par le contrat

Objectif : *L'acte existe dans la nomenclature mais est marqué couvert=false dans refActes.*

```
await Prestation.aggregate([ { $match: { fichierImporteId } }, { $lookup: { from: 'refactes', localField:
'codeActe', foreignField: 'codeActe', pipeline: [{ $match: { compagnieId, couvert: false } }], as:
'acteRef' }}, { $unwind:{ path:'$acteRef', preserveNullAndEmpty:false }}, { $project: {
matriculeBeneficiaire:1, nomBeneficiaire:1, prenomBeneficiaire:1, nomAssurePrincipal:1,
prenomAssurePrincipal:1, matriculeAssurePrincipal:1, codeActe:1, libelleActe:'$acteRef.libelle',
famillePrestation:1, dateSoins:1, montantPaye:1 }} ]);
```

7.3 — Acte incompatible avec le profil du bénéficiaire

Objectif : *Croise refActes.profilBeneficiaire avec le sexe/âge de l'assuré dans refAssures. Ex: acte gynécologique pour un homme, acte pédiatrique pour un adulte.*

```
await Prestation.aggregate([ { $match: { fichierImporteId } }, { $lookup: { from: 'refactes', localField:
'codeActe', foreignField: 'codeActe', pipeline:[{ $match:{ compagnieId, profilBeneficiaire:{
$nin:['tous',null] } } }], as: 'acteRef' }}, { $unwind:{ path:'$acteRef', preserveNullAndEmpty:false }},
{ $lookup: { from: 'refassures', localField: 'matriculeBeneficiaire', foreignField: 'matricule',
pipeline:[{ $match:{ compagnieId } }], as: 'assureRef' }}, { $unwind:{ path:'$assureRef',
preserveNullAndEmpty:false }}, // 4 cas d'incompatibilité : femme/homme/enfant/adulte { $match: { $or: [
{ 'acteRef.profilBeneficiaire':'femme', 'assureRef.sexe':{ $in:['M','masculin','homme'] } }, {
'acteRef.profilBeneficiaire':'homme', 'assureRef.sexe':{ $in:['F','feminin','femme'] } }, {
'acteRef.profilBeneficiaire':'enfant', $expr:{ $gte:[{ $dateDiff:{ startDate:'$assureRef.dateNaissance',
endDate:'$dateSoins', unit:'year'} }, 18] } }, { 'acteRef.profilBeneficiaire':'adulte', $expr:{ $lt:[{
$dateDiff:{ startDate:'$assureRef.dateNaissance', endDate:'$dateSoins', unit:'year'} }, 18] } } ]}} ]);
```

7.4 — Acte chirurgical sans hospitalisation

Objectif : *Acte avec necessiteHospitalisation=true dans refActes, mais sans séjour HOSPITALISATION pour ce bénéficiaire dans une fenêtre de ±7 jours.*

```
await Prestation.aggregate([ { $match: { fichierImporteId } }, { $lookup: { from:'refactes',
localField:'codeActe', foreignField:'codeActe', pipeline:[{ $match:{ compagnieId,
necessiteHospitalisation:true } }], as:'acteRef' }}, { $unwind:{ path:'$acteRef',
preserveNullAndEmpty:false }}, // Chercher une hospitalisation correspondante (±7 jours) { $lookup: {
from:'prestations', let:{ mat:'$matriculeBeneficiaire', ds:'$dateSoins', fid:'$fichierImporteId' },
pipeline:[{ $match:{ $expr:{ $and:[ { $eq:['$fichierImporteId','$$fid'] }, {
$eq:['$matriculeBeneficiaire','$$mat'] }, { $eq:['$famillePrestation','HOSPITALISATION'] }, {
$gte:['$dateSoins', { $subtract:['$$ds',7*24*3600*1000] }] }, { $lte:['$dateSoins', {
$add:['$$ds',7*24*3600*1000] }] } ]}}}], as:'hospit' }}, // Aucune hospitalisation correspondante {
$match: { hospit: { $size: 0 } } } ]);
```

7.5 — Répétition suspecte du même acte

Objectif : *Même codeActe répété plus de nbMax fois (défaut: 5) pour le même bénéficiaire sur une fenêtre de nbJours jours (défaut: 30).*

```
// nbMax=5 occurrences, nbJours=30 jours await Prestation.aggregate([ { $match: { fichierImporteId } }, { $group: { _id: { matriculeBeneficiaire: '$matriculeBeneficiaire', codeActe: '$codeActe' }, count: { $sum: 1 }, montantTotal: { $sum: '$montantPaye' }, dateMin: { $min: '$dateSoins' }, dateMax: { $max: '$dateSoins' }, nomBeneficiaire: { $first: '$nomBeneficiaire' }, prenomBeneficiaire: { $first: '$prenomBeneficiaire' }, nomAssurePrincipal: { $first: '$nomAssurePrincipal' }, prenomAssurePrincipal: { $first: '$prenomAssurePrincipal' }, matriculeAssurePrincipal: { $first: '$matriculeAssurePrincipal' }, ids: { $push: '$_id' } }}, { $match: { count: { $gt: nbMax } } }, // Vérifier que les occurrences sont dans la fenêtre de nbJours { $match: { $expr: { $lte: [ { $dateDiff:{ startDate:'$dateMin', endDate:'$dateMax', unit:'day' } }, nbJours ]}}} ]);
```

CATÉGORIE 8 — CONTRÔLES TEMPORELS

INCONDITIONNEL

Vérifie la cohérence temporelle des soins : dates futures, hors période contractuelle, écarts suspects et prestations tardives hors période d'audit.

8.1 — Date de soins dans le futur

Objectif : *dateSoins strictement postérieure à la date d'importation du fichier.*

```
const maintenant = new Date(); await Prestation.find({ fichierImporteId, dateSoins: { $gt: maintenant } }).select('numeroPrestation matriculeAssurePrincipal nomAssurePrincipal prenomAssurePrincipal matriculeBeneficiaire nomBeneficiaire prenomBeneficiaire dateSoins montantPaye');
```

8.2 — Date hors période contractuelle

Objectif : *dateSoins avant dateEffet ou après dateEcheance du contrat dans refContrats.*

```
await Prestation.aggregate([ { $match: { fichierImporteId } }, { $lookup: { from: 'refcontrats', localField: 'numeroContrat', foreignField: 'numeroContrat', pipeline: [{ $match: { compagnieId } }], as: 'contratRef' }}, { $unwind:{ path:'$contratRef', preserveNullAndEmpty:false }}, // Hors fenêtre contractuelle { $match: { $or: [ { $expr:{ $lt:['$dateSoins','$contratRef.dateEffet'] } }, { $expr:{ $gt:['$dateSoins','$contratRef.dateEcheance'] } } ]}}, { $project: { matriculeBeneficiaire:1, nomBeneficiaire:1, prenomBeneficiaire:1, nomAssurePrincipal:1, prenomAssurePrincipal:1, matriculeAssurePrincipal:1, dateSoins:1, montantPaye:1, numeroContrat:1, dateEffet: '$contratRef.dateEffet', dateEcheance:'$contratRef.dateEcheance' }} ]);
```

8.3 — Écart excessif dateSoins / dateComptable

Objectif : *Écart en valeur absolue > joursMax (défaut: 90 jours) entre dateSoins et dateComptable. Indique une rétro-facturation suspecte.*

```
// joursMax = 90 jours (configurable) await Prestation.aggregate([ { $match: { fichierImporteId, dateSoins:{ $ne:null }, dateComptable:{ $ne:null } }}, { $addFields: { ecartJours: { $abs: { $dateDiff: { startDate: '$dateSoins', endDate: '$dateComptable', unit: 'day' }}} }}, { $match: { ecartJours: { $gt: joursMax } } }, { $project: { matriculeBeneficiaire:1, nomBeneficiaire:1, prenomBeneficiaire:1, nomAssurePrincipal:1, prenomAssurePrincipal:1, matriculeAssurePrincipal:1, dateSoins:1, dateComptable:1, ecartJours:1, montantPaye:1 }} ]);
```

8.4 — Prestations antérieures à la naissance

Objectif : *dateSoins < dateNaissance du bénéficiaire dans refAssures.*

```
await Prestation.aggregate([ { $match: { fichierImporteId } }, { $lookup: { from: 'refassures', localField: 'matriculeBeneficiaire', foreignField: 'matricule', pipeline:[{ $match:{ compagnieId, dateNaissance:{ $ne:null } } }], as: 'assureRef' }}, { $unwind:{ path:'$assureRef', preserveNullAndEmpty:false }}, { $match: { $expr:{ $lt:['$dateSoins','$assureRef.dateNaissance'] } }}, { $project: { matriculeBeneficiaire:1, nomBeneficiaire:1, prenomBeneficiaire:1, nomAssurePrincipal:1, prenomAssurePrincipal:1, matriculeAssurePrincipal:1, dateSoins:1, montantPaye:1, dateNaissance:'$assureRef.dateNaissance' }} ]);
```

8.5 — Prestations tardives (hors période d'audit)

Objectif : *dateSoins hors de la fenêtre [periodeAuditDebut, periodeAuditFin] définie par l'auditeur. Ces lignes sont extraites, rejetées du calcul et notifiées au gestionnaire.*

```
// Récupérer la période d'audit du fichier const fichier = await FichierImporte.findById(fichierImporteId) .select('periodeAuditDebut periodeAuditFin'); const { periodeAuditDebut, periodeAuditFin } = fichier; // Requête : dateSoins hors de la période d'audit const lignes = await Prestation.find({ fichierImporteId, $or: [ { dateSoins: { $lt: periodeAuditDebut } }, { dateSoins: { $gt: periodeAuditFin } } ] }).select('numeroPrestation matriculeAssurePrincipal nomAssurePrincipal prenomAssurePrincipal matriculeBeneficiaire nomBeneficiaire prenomBeneficiaire dateSoins montantPaye codeActe prestataire'); // Marquer les prestations tardives comme rejetées await Prestation.updateMany( { _id: { $in: lignes.map(l => l._id) } }, { $set: { estTardive: true, estRejetee: true } } );
```

CATÉGORIE 9 — COHÉRENCE CONTRACTUELLE

CONDITIONNEL | *Condition : Table refContrats renseignée*

Vérifie la cohérence des références contractuelles dans le fichier : contrats actifs, gestionnaire mandaté, assureur et souscripteur.

9.1 — Contrat inexistant

Objectif : *Le numeroContrat du fichier est absent de la table refContrats de la compagnie.*

```
await Prestation.aggregate([ { $match: { fichierImporteId, numeroContrat:{ $ne:null } }}, { $lookup: {
from: 'refcontrats', localField: 'numeroContrat', foreignField: 'numeroContrat', pipeline:[{ $match:{
compagnieId } }], as: 'contratRef' }}, { $match: { contratRef: { $size: 0 } } }, { $project: {
matriculeBeneficiaire:1, nomBeneficiaire:1, prenomBeneficiaire:1, nomAssurePrincipal:1,
prenomAssurePrincipal:1, matriculeAssurePrincipal:1, numeroContrat:1, dateSoins:1, montantPaye:1 }} ]);
```

9.2 — Gestionnaire non mandaté

Objectif : *Le contrat est attribué à un autre gestionnaire dans refContrats.*

```
await Prestation.aggregate([ { $match: { fichierImporteId, numeroContrat:{ $ne:null } }}, { $lookup: {
from: 'refcontrats', localField: 'numeroContrat', foreignField: 'numeroContrat', pipeline:[{ $match:{
compagnieId, // Mandaté à un AUTRE gestionnaire gestionnaireId:{ $ne: gestionnaireId } }}], as:
'contratRef' }}, { $unwind:{ path:'$contratRef', preserveNullAndEmpty:false }}, { $project: {
matriculeBeneficiaire:1, nomBeneficiaire:1, prenomBeneficiaire:1, nomAssurePrincipal:1,
prenomAssurePrincipal:1, matriculeAssurePrincipal:1, numeroContrat:1, dateSoins:1, montantPaye:1,
gestionnaireMandate:'$contratRef.gestionnaireId' }} ]);
```

9.3 — Assureur incohérent

Objectif : *Le codeAssureur/assureur dans le fichier ne correspond pas au code de la compagnie destinataire.*

```
// Récupérer le code de la compagnie const compagnie = await Compagnie.findById(compagnieId)
.select('code nom'); // Chercher les lignes avec un codeAssureur non conforme const lignes = await
Prestation.find({ fichierImporteId, codeAssureur: { $ne: null, // Expression régulière insensible à la
casse $not: new RegExp(compagnie.code, 'i') } }).select('numeroPrestation matriculeAssurePrincipal
nomAssurePrincipal prenomAssurePrincipal matriculeBeneficiaire nomBeneficiaire prenomBeneficiaire
codeAssureur assureur dateSoins montantPaye');
```

9.4 — Souscripteur incohérent

Objectif : *Le souscripteur dans le fichier diffère du souscripteur dans refContrats pour le même numéro de contrat.*

```
await Prestation.aggregate([ { $match: { fichierImporteId, numeroContrat:{ $ne:null } }}, { $lookup: {
from: 'refcontrats', localField: 'numeroContrat', foreignField: 'numeroContrat', pipeline:[{ $match:{
compagnieId } }], as: 'contratRef' }}, { $unwind:{ path:'$contratRef', preserveNullAndEmpty:false }}, //
Comparaison insensible à la casse { $match: { $expr: { $and: [ { $ne:['$contratRef.souscripteur', null]
}, { $ne:[ { $toLower:{ $trim:{ input:'$souscripteur' } } }, { $toLower:{ $trim:{
input:'$contratRef.souscripteur' } } } ]} ]}}}, { $project: { matriculeBeneficiaire:1,
nomBeneficiaire:1, prenomBeneficiaire:1, nomAssurePrincipal:1, prenomAssurePrincipal:1,
matriculeAssurePrincipal:1, souscripteur:1, numeroContrat:1, dateSoins:1, montantPaye:1,
souscripteurRef:'$contratRef.souscripteur' }} ]);
```

CATÉGORIE 10 — ANALYSES STATISTIQUES

INCONDITIONNEL

Analyses avancées pour détecter des schémas frauduleux non visibles ligne par ligne : scores de risque, loi de Benford, comparaisons temporelles et clustering d'anomalies.

10.1 — Score de risque par prestataire

Objectif : *Calcule un indice de suspicion (0-100) par prestataire basé sur le montant moyen, le ratio actes/bénéficiaires et le volume total.*

```
await Prestation.aggregate([ { $match: { fichierImporteId } }, { $group: { _id: '$codePrestataire', nomPrestataire: { $first: '$prestataire' }, nbActes: { $sum: 1 }, montantTotal: { $sum: '$montantPaye' }, montantMoyen: { $avg: '$montantPaye' }, beneficiairesDistincts: { $addToSet:'$matriculeBeneficiaire'}, acteDistincts: { $addToSet: '$codeActe' } }}, { $addFields: { nbBeneficiaires: { $size: '$beneficiairesDistincts' }, nbActesDistincts: { $size: '$acteDistincts' }, // Ratio actes par bénéficiaire ratioActesBenef: { $cond:[ { $gt:[{ $size:'$beneficiairesDistincts' }, 0] }, { $divide:['$nbActes', { $size:'$beneficiairesDistincts' }] }, 0 ]} }}, { $sort: { montantTotal: -1 } } ]); // Score calculé en JS : montantMoyen vs moyenne globale, // ratioActesBenef, volume (top 10%), diversité actes.
```

10.2 — Score de risque par bénéficiaire

Objectif : *Calcule un indice de suspicion (0-100) par bénéficiaire basé sur le nombre d'actes, le nombre de jours de soins et le montant total.*

```
await Prestation.aggregate([ { $match: { fichierImporteId } }, { $group: { _id: '$matriculeBeneficiaire', nomBeneficiaire: { $first: '$nomBeneficiaire' }, prenomBeneficiaire: { $first: '$prenomBeneficiaire' }, nomAssurePrincipal: { $first: '$nomAssurePrincipal' }, prenomAssurePrincipal: { $first: '$prenomAssurePrincipal' }, matriculeAssurePrincipal: { $first: '$matriculeAssurePrincipal' }, nbActes: { $sum: 1 }, montantTotal: { $sum: '$montantPaye' }, montantMoyen: { $avg: '$montantPaye' }, prestatairesDistincts:{ $addToSet:'$codePrestataire' }, datesDistinctes: { $addToSet: { $dateToString:{ format:'%Y-%m-%d', date:'$dateSoins' } }} }}, { $addFields: { nbPrestataires: { $size:'$prestatairesDistincts' }, nbJoursSoins: { $size:'$datesDistinctes' } }}, { $sort: { montantTotal: -1 } } ]); // Score JS : nbActes/nbJours, montantTotal vs moyenne, // nbPrestataires, montantMoyen vs moyenne globale.
```

10.3 — Comparaison mois par mois

Objectif : *Compare le montant total du fichier courant avec la moyenne des 3 derniers fichiers du même gestionnaire. Alerte si variation > ±50%.*

```
// Récupérer les 3 derniers fichiers analysés du gestionnaire const fichiersAnt = await FichierImporte.find({ gestionnaireId, _id: { $ne: fichierImporteId }, statut: 'analysé' }) .sort({ periodeAuditDebut: -1 }) .limit(3) .select('periodeAuditDebut montantTotalReclame'); // Calcul de la variation const moyennePrecedente = fichiersAnt.reduce( (acc, f) => acc + f.montantTotalReclame, 0 ) / fichiersAnt.length; const variation = ((montantCourant - moyennePrecedente) / moyennePrecedente) * 100; // Alerte si |variation| > 50% if (Math.abs(variation) > 50) { /* créer anomalie */ }
```

10.5 — Règle de Benford sur les montants

Objectif : *Applique la loi de Benford sur les premiers chiffres des montantPaye. Un écart > 5 points sur un chiffre indique une manipulation probable.*

```
// Distribution attendue de Benford (en %) const BENFORD = { '1':30.1,'2':17.6,'3':12.5,'4':9.7,'5':7.9, '6':6.7, '7':5.8, '8':5.1, '9':4.6 }; // Récupérer tous les montants > 0 const montants = await Prestation.find( { fichierImporteId, montantPaye:{ $gt:0 } }, { montantPaye:1 } ); // Compter les premiers chiffres const freq = {'1':0,'2':0,'3':0,'4':0,'5':0, '6':0,'7':0,'8':0,'9':0}; montants.forEach(p => { const d = String(Math.floor(p.montantPaye))[0]; if (freq[d] !== undefined) freq[d]++; }); // Calculer les écarts et détecter les anomalies (>5%) const anomalies = Object.entries(freq).filter(([d, count]) => Math.abs((count/montants.length)*100 - BENFORD[d]) > 5 );
```

10.6 — Clustering des comportements suspects

Objectif : *Regroupe les anomalies déjà détectées par prestataire. Un prestataire impliqué dans ≥ 3 types d'anomalies différents est identifié comme cluster de fraude organisée.*

```
// Récupérer toutes les anomalies du fichier avec leur prestataire const anomaliesExistantes = await Anomalie.find({ fichierImporteId }) .select('code categorie detail'); // Regrouper par codePrestataire const parPrestataire = {}; anomaliesExistantes.forEach(a => { const code = a.detail?.codePrestataire; if (!code) return; if (!parPrestataire[code]) parPrestataire[code] = []; parPrestataire[code].push(a.code); }); // Prestataires avec >= 3 types d'anomalies différents = cluster const clusters = Object.entries(parPrestataire) .filter(([, codes]) => new Set(codes).size >= 3) .map(([code, codes]) => ({ codePrestataire: code, typesAnomalies: [...new Set(codes)], nbAnomalies: codes.length }));
```

ANNEXE — STRUCTURE D'UNE ANOMALIE

Toutes les anomalies détectées par Crystal sont stockées dans la collection **anomalies** avec la structure suivante. Le champ **detail** est propre à chaque type d'anomalie et contient les informations spécifiques (noms, matricules, valeurs comparées).

Champ	Type	Description
prestationId	ObjectId	Référence à la prestation concernée (null si anomalie globale)
fichierImporteId	ObjectId	Fichier de soins source
categorie	String	INTEGRITE | BENEFICIAIRE | PRESTATAIRE | DOUBLON | FREQUENCE | MONTANT | ACTE |
code	String	Code unique du contrôle (ex: DOUBLON_FONCTIONNEL, PRESTATION_TARDIVE)
description	String	Message lisible décrivant l'anomalie avec les valeurs observées
gravite	String	critique | élevé | moyen | faible
montantImpacte	Number	Montant financier de l'anomalie (en FCFA)
detail	Object	Données spécifiques : matricules, noms, valeurs comparées, écarts
dateDetection	Date	Date/heure de détection automatique

Niveaux de gravité

Niveau	Code	Signification	Action recommandée
CRITIQUE	critique	Fraude probable	Bloquer le paiement, investigation immédiate
ÉLEVÉ	élevé	Anomalie sérieuse	Investigation obligatoire avant paiement
MOYEN	moyen	Incohérence détectée	Vérification recommandée
FAIBLE	faible	Observation	Surveiller l'évolution

CRYSTAL — ARIEL INDUSTRIES SARL — Documentation technique confidentielle — Mai 2026